

BÚSQUEDA BINARIA

La **búsqueda binaria** es un algoritmo eficiente para encontrar un elemento en una lista **ordenada**. En lugar de buscar secuencialmente, la búsqueda binaria divide la lista en mitades, reduciendo significativamente el número de comparaciones que se necesitan para encontrar el elemento deseado.

Descripción del Algoritmo

Supón que tienes una lista ordenada y deseas encontrar un elemento en particular. La búsqueda binaria sigue estos pasos:

1. **Dividir la lista a la mitad:** Compara el elemento buscado con el elemento que está en el medio de la lista.
2. **Si el elemento es igual al valor del medio,** has encontrado el elemento y el algoritmo termina.
3. **Si el elemento es menor que el valor del medio** descarta la mitad superior de la lista y busca en la mitad inferior.
4. **Si el elemento es mayor que el valor del medio** descarta la mitad inferior de la lista y busca en la mitad superior.
5. **Repetir el proceso** con la mitad correspondiente hasta encontrar el elemento o hasta que no haya más elementos que buscar.

La eficiencia de la búsqueda binaria proviene del hecho de que en cada paso reduce el espacio de búsqueda a la mitad, lo que la hace mucho más rápida que una búsqueda lineal.

Condición necesaria: La lista debe estar ordenada previamente para que la búsqueda binaria funcione.

Complejidad del Algoritmo

- **Mejor caso:** $O(1)$ – Si el elemento está en la posición media en el primer intento.
- **Peor caso:** $O(\log n)$ – En cada paso, la lista se reduce a la mitad, por lo que el número de comparaciones necesarias crece logarítmicamente con el tamaño de la lista.

Ejemplo en Java

A continuación, te muestro un ejemplo de cómo implementar la búsqueda binaria en Java.

```
public class BusquedaBinaria {

    // Método para realizar la búsqueda binaria
    public static int busquedaBinaria(int[] arr, int valorBuscado) {
        int inicio = 0;
        int fin = arr.length - 1;

        while (inicio <= fin) {
            int medio = inicio + (fin - inicio) / 2; // Encontrar el
            índice medio

            // Verificar si el valor buscado está en el medio
            if (arr[medio] == valorBuscado) {
                return medio; // Elemento encontrado
            }

            // Si el valor buscado es mayor, ignorar la mitad
            izquierda
            if (arr[medio] < valorBuscado) {
                inicio = medio + 1;
            } else {
                // Si el valor buscado es menor, ignorar la mitad
                derecha
                fin = medio - 1;
            }

            // El valor no está presente en el arreglo
            return -1;
        }

        public static void main(String[] args) {
            int[] arreglo = {2, 3, 4, 10, 40}; // El arreglo debe estar
            ordenado
            int valorBuscado = 10;

            // Llamar al método de búsqueda binaria
            int resultado = busquedaBinaria(arreglo, valorBuscado);

            if (resultado == -1) {
                System.out.println("Elemento no encontrado");
            } else {
                System.out.println("Elemento encontrado en el índice: " +
                resultado);
            }
        }
    }
}
```

Explicación del Código:

- **Entrada:** Un arreglo de enteros **ordenado** y un valor que quieres encontrar en ese arreglo.
- **Proceso:** El algoritmo divide repetidamente el arreglo por la mitad y compara el valor buscado con el valor en la posición media. Si el valor buscado es menor o mayor, ajusta los límites (inicio y fin) para restringir la búsqueda a la mitad correspondiente.
- **Salida:** Si el valor se encuentra en el arreglo, el método retorna su índice. Si no se encuentra, retorna -1.

Ejemplo de Ejecución:

Para el arreglo {2, 3, 4, 10, 40} y el valor buscado 10, la búsqueda binaria encontrará el valor en el índice 3.

Comparación con la Búsqueda Lineal

- **Búsqueda lineal:** Busca el elemento recorriendo la lista desde el principio hasta el final, con una complejidad $O(n)$.
- **Búsqueda binaria:** Divide la lista en partes y tiene una complejidad $O(\log n)$, lo que la hace mucho más eficiente para listas grandes.

Casos en los que se usa la Búsqueda Binaria

1. **Grandes volúmenes de datos ordenados:** Cuando trabajas con listas o arreglos ordenados y necesitas encontrar un elemento rápidamente.
2. **Rango de valores continuos:** Puedes usar la búsqueda binaria en problemas donde el conjunto de soluciones es continuo (por ejemplo, buscar una raíz cuadrada de un número).
3. **Bases de datos:** La búsqueda binaria es útil en bases de datos para encontrar registros de manera eficiente cuando los datos están indexados.

Conclusión

La **búsqueda binaria** es un algoritmo eficiente que reduce la cantidad de comparaciones necesarias para encontrar un elemento en una lista ordenada. Funciona mejor en comparación con una búsqueda lineal en grandes listas, ya que su complejidad es $O(\log n)$ en lugar de $O(n)$.

¿Por qué $O(\log n)$?

Para entender por qué la búsqueda binaria tiene una complejidad de $O(\log n)$, veamos el proceso paso a paso:

1. **División en mitades:** Al comenzar, tienes una lista de n elementos. Lo primero que haces es comparar el elemento en el medio con el valor que estás buscando. Esta comparación es $O(1)$ (constante), porque solo haces una comparación.
2. **Descartar la mitad:** Si el valor buscado no es igual al elemento en el medio, puedes descartar **la mitad** de la lista:
 - Si el valor buscado es menor, buscas en la mitad izquierda.
 - Si el valor buscado es mayor, buscas en la mitad derecha.

Como descartas la mitad de la lista en cada paso, el espacio de búsqueda se reduce a la mitad en cada iteración.

3. **Número de divisiones:** En cada iteración, la cantidad de elementos que necesitas revisar se reduce a la mitad. El número de veces que puedes dividir la lista antes de que quede solo un elemento es aproximadamente el **logaritmo en base 2 del tamaño de la lista ($\log_2 n$)**.
4. **Logaritmo base 2:** La cantidad de pasos necesarios para reducir la lista de tamaño n a 1 es el número de veces que puedes dividir el número n entre 2 antes de llegar a 1. Esto es exactamente lo que el logaritmo en base 2 representa.

Formalmente, si comienzas con una lista de tamaño n y reduces a la mitad el tamaño de la lista en cada paso, el número de pasos requeridos será:

$$\log_2 16 = 4$$

Entonces, el número de pasos que tomará encontrar el elemento o determinar que no está presente en la lista es proporcional a:

$$\log_2 n$$

Ejemplo:

Imagina que tienes una lista de **16 elementos** y estás buscando un número usando búsqueda binaria:

1. **Primer paso:** Comienzas comparando con el elemento en el medio. Tienes 16 elementos, así que revisas el elemento en el índice 8. Ahora, **tienes dos posibilidades:** si el valor buscado es menor que este, descartas la mitad derecha (los últimos 8 elementos); si es mayor, descartas la mitad izquierda (los primeros 8 elementos). **Tamaño restante: 8 elementos.**
2. **Segundo paso:** Ahora buscas en la mitad restante. Tienes 8 elementos. Nuevamente, comparas el valor en el medio (índice 4 en este caso). **Tamaño restante: 4 elementos.**
3. **Tercer paso:** Te quedas con 4 elementos. Comparas el valor en el medio. **Tamaño restante: 2 elementos.**
4. **Cuarto paso:** Ahora comparas los últimos 2 elementos. Si no encuentras el valor, te queda solo 1 elemento. **Tamaño restante: 1 elemento.**

En este ejemplo, para reducir el espacio de búsqueda de 16 elementos a 1, tomaste **4 pasos**. Y si lo calculas:

$$\log_2 16 = 4$$

Generalizando

Si tienes **n** elementos, el número de pasos en la búsqueda binaria es aproximadamente:

$$\log_2 n$$

Es por eso que la complejidad de tiempo de la búsqueda binaria se expresa como **$O(\log n)$** , lo que indica que el tiempo de ejecución del algoritmo crece logarítmicamente en función del tamaño de la lista.

Nota: La **búsqueda binaria** en arreglos de tamaño impar sigue siendo **$O(\log n)$** , y aunque los cálculos exactos para el logaritmo pueden no dar un número entero, redondeamos hacia arriba para contar el número de pasos reales. En este caso, tanto un arreglo de 15 elementos (impar) como uno de 16 elementos (par) requieren aproximadamente **4 pasos** para reducirse a un solo elemento. Por lo tanto, aunque el tamaño del arreglo sea impar, el comportamiento logarítmico de la búsqueda binaria no cambia, y sigue dividiendo el espacio de búsqueda de manera similar.

Ejemplo: Búsqueda Binaria en un Arreglo de 15 Elementos

Imaginemos que queremos buscar un valor en un arreglo de **15 elementos**: {1, 2, 3, 4, 5, 6, 7, 8, 9, 10, 11, 12, 13, 14, 15}. El proceso sería el siguiente:

1. **Paso 1:** El valor medio en un arreglo de tamaño 15 es el elemento en el índice 7 (recuerda que los índices empiezan en 0), que es el número 8.
 - Ahora, comparamos el valor buscado con 8.
 - Si el valor buscado es menor, descartamos la mitad derecha del arreglo (elementos mayores que 8).
 - Si el valor buscado es mayor, descartamos la mitad izquierda.
2. **Paso 2:** Si descartamos una mitad, ahora tenemos un subconjunto de **7 elementos**. El nuevo valor medio está en el índice 3 (dentro de los 7 elementos), y es el número 4. De nuevo, repetimos el proceso de comparación.
3. **Paso 3:** Si descartamos otra mitad, nos quedan **3 elementos**. El nuevo valor medio es el elemento en el índice 1, que es el número 2.
4. **Paso 4:** Finalmente, si descartamos otra mitad, nos queda solo **1 elemento** en el subconjunto.

Resumen de los Pasos

- En un arreglo de 15 elementos, después de 4 divisiones del arreglo, solo queda un elemento.
- El número de pasos es 4, igual que el número de divisiones en un arreglo de tamaño par de 16 elementos.

Relación con el Logaritmo Base 2

El número de pasos sigue estando relacionado con el **logaritmo en base 2**, ya que en cada paso reduces el espacio de búsqueda a la mitad. En el caso de un arreglo de 15 elementos:

$$\log_2(15) \approx 3.91$$

Dado que no se puede hacer media división en el mundo real (no hay mitad de un elemento), redondeamos hacia arriba a **4 pasos**.

Caso General 1: Buscar un Elemento en una Lista Ordenada

Dada una lista **ordenada** de elementos y un valor **buscado**, podemos seguir los pasos de la búsqueda binaria:

Caso 1: Valor en el Medio

Si el valor buscado es igual al valor medio en la primera comparación, se encuentra en el medio y el algoritmo termina.

Ejemplo General:

- **Lista:** {A, B, C, D, E}
- **Buscado:** C
- **Proceso:** Comparamos con el valor medio (índice 2) que es C, lo encontramos inmediatamente.

Caso 2: Valor en la Mitad Derecha

Si el valor buscado es mayor que el valor medio, ajustamos el rango de búsqueda a la mitad derecha.

Ejemplo General:

- **Lista:** {A, B, C, D, E, F, G, H, I}
- **Buscado:** H
- **Proceso:**
 1. Comparamos con el valor medio (índice 4), que es E. Como $H > E$, ajustamos el rango a {F, G, H, I}.

2. Comparamos con el nuevo valor medio (índice 6), que es G. Como $H > G$, ajustamos el rango a $\{H, I\}$.
3. Comparamos con el nuevo valor medio (índice 7), que es H. Lo encontramos.

Caso 3: Valor en la Mitad Izquierda

Si el valor buscado es menor que el valor medio, ajustamos el rango de búsqueda a la mitad izquierda.

Ejemplo General:

- **Lista:** $\{A, B, C, D, E, F, G, H, I, J, K\}$
- **Buscado:** B
- **Proceso:**
 1. Comparamos con el valor medio (índice 5), que es F. Como $B < F$, ajustamos el rango a $\{A, B, C, D, E\}$.
 2. Comparamos con el nuevo valor medio (índice 2), que es C. Como $B < C$, ajustamos el rango a $\{A, B\}$.
 3. Comparamos con el nuevo valor medio (índice 0), que es A. Como $B > A$, ajustamos el rango a $\{B\}$.
 4. Lo encontramos.

Caso 4: Valor No Está en la Lista

Si el valor buscado no está en la lista, después de ajustar el rango varias veces, la búsqueda se detiene cuando $\text{inicio} > \text{fin}$.

Ejemplo General:

- **Lista:** {1, 2, 3, 4, 5, 6, 7, 8, 9}
- **Buscado:** 10
- **Proceso:**
 1. Comparamos con el valor medio (índice 4), que es 5. Como $10 > 5$, ajustamos el rango a {6, 7, 8, 9}.
 2. Comparamos con el nuevo valor medio (índice 6), que es 7. Como $10 > 7$, ajustamos el rango a {8, 9}.
 3. Comparamos con el nuevo valor medio (índice 7), que es 8. Como $10 > 8$, ajustamos el rango a {9}.
 4. Comparamos con el último valor 9. Como $10 > 9$, ajustamos $\text{inicio} > \text{fin}$ y terminamos.

Caso General 2: Tamaño Par e Impar

Lista de Tamaño Impar

Al dividir una lista de tamaño impar, siempre hay un valor exacto en el medio.

Ejemplo General:

- **Lista:** {A, B, C, D, E} (tamaño impar).
- **El valor medio** está en el índice 2, que es C.

Lista de Tamaño Par

Al dividir una lista de tamaño par, se elige el valor medio como el elemento que esté entre el primer y segundo bloque de la lista dividida.

Ejemplo General:

- **Lista:** {A, B, C, D, E, F} (tamaño par).
- **El valor medio** está en el índice 2 (depende de la implementación si elige el primero o el segundo en el medio).